TALK-TO-DATA (T2D)

Phase 3: Governed Data Foundation

Daniel Brulé
LinkedIn - GitHub
Version 1.0 - June 2026

Table of Contents

1. Executive summary

Phase 3 turns the answerable questions from Phase 2 into governed data foundations that a Talk-to-Data system can safely query.

The purpose of this phase is not to build the conversational interface. It is to create the controlled data layer beneath it: approved sources, curated views or models, implemented metrics, usable dimensions, safe joins, standard filters, security controls, caveats, metadata and quality checks. Without this layer, the T2D system may generate valid SQL and fluent answers while still producing numbers that are wrong, misleading, unauthorised or impossible to explain.

The depth of Phase 3 should match the delivery intent. For a constrained POC, the foundation may be deliberately narrow: limited assets, limited users, clear caveats and enough control to support safe learning. For an MVP, pilot or production path, the standard is higher: logic must be tested, access and exposure controls enforceable, caveats documented, quality limits understood and ownership explicit.

The central discipline is to **move business logic out of prompts and into governed assets**. Metric definitions, date logic, exclusions, dimensional hierarchies, join paths and access rules should not depend on the model inferring meaning from loose documentation. Wherever possible, they should be encoded, tested and owned in the data foundation.

Phase 3 should start from the Phase 2 answerability matrix. Only questions, sources, metrics and dimensions assessed as suitable for the current delivery stage should move into foundation build. Unresolved definitions, unsafe joins, weak quality, unclear ownership or incomplete access controls should remain in remediation, backlog or exclusion.

By the end of Phase 3, stakeholders should be able to decide whether the governed foundation is fit for the intended next step. For a POC, that means confirming whether a narrow, controlled foundation is sufficient to test feasibility and estimate the effort required for MVP, pilot or production. For an MVP or pilot, it means confirming whether the data layer is reliable, controlled and explainable enough to support repeated use by the target users.

Phase 3 should also prepare the foundation artefacts for downstream orchestration. Metrics, dimensions, joins, filters, caveats, access rules and quality limits must be structured so that Phase 4 can retrieve, interpret and apply them consistently. The format may vary by delivery stage, from a structured spreadsheet in a POC to a governed catalogue, semantic layer, repository or API in production, but the metadata contract must be explicit enough to support safe query generation, validation and answer explanation.

The main output is a **governed foundation pack**: approved queryable assets, metric and dimension implementations, join and filter rules, security controls, quality checks, metadata, ownership records, known limitations and expected foundation run-cost implications. For a POC, this supports safe learning and helps size the effort required for MVP, pilot or production. For MVP, pilot or production, it provides the controlled foundation required for reliable conversational analytics.

2. Phase overview

Phase 3 builds the governed data foundation for the scoped Talk-to-Data use case. It turns Phase 2 answerability decisions into controlled, queryable assets that the T2D can safely use.

The phase should start from the agreed scope, not restart discovery. However, implementation may still expose issues such as unsafe joins, contested logic, missing data, weak ownership or access constraints. When that happens, the team should narrow, remediate, defer or exclude the affected question rather than force it into the foundation.

The key discipline is to avoid building the assistant directly on raw or loosely understood data. T2D requires a controlled query surface: approved assets, implemented business logic, safe joins, standard filters, security controls, quality checks, caveats and metadata.

For a POC, this foundation may be narrow and partly temporary. For an MVP, pilot or production path, it must be stronger, repeatable and owned.

2.1. Objective

The objective of Phase 3 is to implement the controlled data foundation required for the agreed T2D scope. It should confirm and implement:

- **Queryable assets:** approved tables, views, models, marts, APIs, semantic-layer assets.
- **Business logic:** implemented metrics, dimensions, hierarchies, date, exclusions, caveats.
- **Query safety:** approved grains, joins, aggregation rules, filters and row limits.
- **Security controls:** row-level access, column restrictions, masking, aggregation limits, inference-risk controls and audit requirements.
- **Foundation controls:** metadata, ownership, stable IDs, versions, usage rules, quality and freshness checks, monitoring needs, run-cost implications and known limitations.

The output is a **governed foundation pack** that can be used by architecture, orchestration, prototype / MVP build, evaluation and security validation.

2.2. Scope of the phase

Phase 3 should remain bounded to the use case, users, questions and delivery maturity agreed in earlier phases. It implements the governed queryable foundation for the current scope; it should not become a broader data-platform transformation.

In scope are the approved assets, metric and dimension logic, joins, filters, security controls, metadata, caveats, quality checks, run costs and ownership records required for safe T2D use.

The test for inclusion is simple: if an item is required for the system to query, explain, control or evaluate the scoped answers safely, it belongs in Phase 3.

Phase 3 should not hide unresolved ambiguity inside a prompt, view or model. If a metric, join, filter or source remains materially contested, the team should narrow, remediate, defer or exclude it.

2.3. Expected duration and level of effort

For a narrow POC, Phase 3 may be completed quickly if the required sources and logic already exist. The work may involve a small number of curated assets, clear caveats and lightweight controls to assess feasibility and estimate the effort required for MVP, pilot or production.

For an MVP or pilot, the work is usually more substantial. The foundation needs to be repeatable, tested, versioned and owned. Metric logic, joins, filters, quality checks, metadata and security rules should be implemented through appropriate engineering controls, including review, deployment discipline and rollback where needed.

The phase should not be judged by the number of assets created. A small, well-controlled foundation with clear change control and predictable run cost is more valuable than a broad foundation that leaves the assistant to improvise definitions, joins, access behaviour or expensive queries.

2.4. Main participants and decision owners

Phase 3 requires close collaboration between data, business, semantic, security and architecture roles. It should not be treated as a purely engineering activity.

Role	Main responsibility in Phase 3
Product owner	Confirms scope, trade-offs and prioritisation.
Business SME / domain owner	Validates implemented business meaning and caveats.
Data owner	Approves source use, quality constraints and data boundaries.
Metric / semantic owner	Approves definitions, hierarchies, synonyms and caveats.
Data architect / analytics engineer	Designs governed assets, joins, grains and reusable query structures; ensures the foundation can scale beyond the first question set where required.
Data engineer	Builds data assets, pipelines, quality checks and freshness controls; ensures implementation is reliable, performant and maintainable.
Security / governance lead	Confirms data sensitivity, user groups, access controls, masking, aggregation limits, audit needs, residual exposure risks and security approval route
AI / solution architect	Confirms the foundation supports orchestration, tool use, safe failure, query performance and scaling assumptions.
Evaluation owner	Ensures the assets support golden questions and regression testing.
Operating owner	Confirms how assets, rules, incidents and change requests will be maintained as usage grows.

3. Foundation decision and delivery implications

Phase 3 should end with a clear decision on whether the governed foundation is fit for the intended next step. The decision should be based on what has been implemented, tested, controlled and owned, not only on what was assumed during framing or readiness assessment.

A proceed decision does not mean the full T2D product is ready to launch. It means the foundation is controlled enough for the next stage: POC learning, MVP build, pilot validation or production hardening.

3.1. Possible Phase 3 outcomes

Outcome	Meaning	Typical trigger
Proceed	Move to architecture, orchestration, build and evaluation with the agreed foundation.	Approved assets, implemented logic, safe joins, security controls and known caveats are strong enough for the intended next step.
Narrow	Continue with a reduced question set, user group, data scope or answer type.	Some foundation elements are reliable, but part of the scope is too broad, fragile or costly to control.
Remediate	Fix specific foundation gaps before broader delivery.	Metric logic, joins, data quality, freshness, metadata, security controls or ownership are not yet strong enough.
Defer	Move some assets, questions, metrics or domains to a later release.	The item may be valuable, but it is not required or not ready for the current POC, MVP or pilot.
Stop	Do not continue this use case as a T2D delivery candidate.	The foundation cannot support trusted answers within viable cost, effort, risk or ownership constraints.

3.2. Minimum conditions to proceed

The team should proceed only when it can answer yes to the following confidence tests:

- **Query surface is bounded:** the assistant has a defined, approved set of assets to query. Broad or raw table access is not a substitute for a governed surface.
- **Business logic is encoded:** metrics, dimensions, date logic, exclusions, caveats and filters are implemented in governed assets, not inferred from documentation or held in prompts.
- **Query structure is safe:** grains, joins, aggregation rules and row limits are controlled well enough to prevent structurally wrong answers.
- **Security controls are implemented, not assumed:** row-level access, masking, aggregation limits, inference-risk controls and audit needs are in place or explicitly constrained for the delivery stage.
- **Quality and cost are understood:** material data-quality risks, freshness gaps, reconciliation limits, performance constraints and run-cost drivers are known, monitored or caveated.
- **Metadata supports explanation:** the system has enough context to surface caveats, describe limitations and fail safely rather than silently.

- **Ownership is real, not nominal:** named owners exist for assets, definitions, quality issues and security rules, and those owners are reachable after the build team moves on.
- **Implementation route is controlled:** the foundation technology, deployment approach, testing, rollback and change-control path are confirmed and being followed.

The evidence bar should match the delivery stage: lightweight for bounded POC learning, materially stronger for MVP, pilot or production.

3.3. Common reasons to narrow, remediate, defer or pause

The team should avoid moving forward unchanged where any of the following apply:

- **The query surface is too broad:** the assistant would need access to too many raw tables, domains or ambiguous sources.
- **Business logic is not implementable:** key metric, filter, hierarchy or date rules are still informal, contested or analyst-dependent.
- **Joins remain unsafe:** valid SQL could still duplicate, omit or misaggregate results because grain and relationships are not controlled.
- **Security controls are incomplete:** authentication exists, but row-level access, masking, aggregation limits, auditability or inference-risk controls are weak.
- **Quality, performance or cost is not controlled:** freshness, reconciliation, query volume, materialisation or monitoring could undermine trust or affordability.
- **Metadata is too thin:** the system cannot explain source, caveat, definition or limitation clearly enough.
- **Ownership is unresolved:** no one owns the asset, metric, caveat, quality issue, security rule or change path after build.

The interface working is not evidence the foundation is ready. A model that improvises metric logic, picks arbitrary joins or papers over missing filters can produce plausible answers that break under scrutiny. That is more dangerous than an obvious failure.

3.4. How Phase 3 shapes later phases

Phase 3 shapes later phases by defining what the assistant may query, which business logic has been implemented, which joins and filters are allowed, which caveats and security controls apply, and which limitations remain unresolved.

These outputs become practical inputs for orchestration design, MVP build, evaluation, security validation, adoption and operations. The detailed handover is consolidated in Section 7.2.

4. Governed data foundation activities overview

Phase 3 is organised around practical build activities. They are connected workstreams, not a rigid sequence. In a narrow POC, activities may be simplified or combined. For an MVP, pilot or production path, each activity needs stronger evidence, implementation control and ownership.

The logic is simple: confirm what the assistant may use, implement the governed query surface, encode business rules, apply security and quality controls, then package the foundation for build, evaluation and validation.

The foundation should favour governed, reusable queryable assets over raw table access or model-generated business logic. This may mean curated views, transformation models, semantic-layer objects, materialised tables, controlled APIs or a hybrid pattern, chosen for control, performance, cost, security, maintainability and reuse.

4.1. Activity sequence

Activity	Main question	Typical output
1. Confirm foundation build scope	Which Phase 2-approved assets, questions, metrics and dimensions will be implemented now?	Build scope, exclusions, ownership and unresolved constraints.
2. Confirm implementation pattern and technology route	How will the foundation be implemented for this scope?	Pattern, technology used, performance assumptions, cost drivers and unresolved platform decisions.
3. Build curated queryable assets	What controlled query surface should the assistant use?	Curated views, models, marts, semantic objects or APIs.
4. Implement metric logic	Which measures can the assistant calculate consistently?	Implemented KPIs, calculations, date logic, exclusions and caveats.
5. Implement dimensions and hierarchies	How should the assistant filter, group and explain results?	Approved dimensions, hierarchies, labels and synonyms.
6. Implement join, grain and aggregation rules	How do we prevent structurally wrong answers?	Approved grains, joins, relationships, aggregation rules and row limits.
7. Implement standard filters and caveats	Which business rules must always be applied or surfaced?	Standard exclusions, time windows, segments, default filters and caveats.
8. Implement security and exposure controls	What can users see, query or infer?	Row/column controls, masking, aggregation limits, audit needs and exposure controls.
9. Add quality, performance, freshness and cost controls	Can the foundation be trusted and afforded in use?	Quality, freshness, reconciliation, performance and cost assumptions.
10. Package foundation for handover	Is the foundation usable by later phases?	Documentation, metadata, owners, known limitations and backlog.

4.2. Cross-activity controls: tests, logs, cost and evidence

Phase 3 activities should produce evidence, not just assets. Automated tests, deployment logs, validation results, cost indicators, alerts and approval records should be captured consistently enough for later phases to understand what was built, what was tested, what failed, what was accepted and what remains unresolved.

For a POC, this evidence may be lightweight. For MVP, pilot or production, it should be centralised, versioned and linked to the assets, metrics, dimensions, filters and controls it validates.

The purpose is traceability and control. When an answer, asset, failure or cost is challenged later, the team should be able to see the implementation, evidence, owner and decision route.

4.3. Activity logic

The activities should reduce the amount of judgement left to the model at runtime. The assistant should not be expected to infer metric definitions, choose arbitrary joins, guess exclusions, bypass access rules or explain caveats from scattered documentation.

Where possible, Phase 3 should encode these rules into governed assets. Documentation and metadata still matter, but they should support interpretation and explanation — not compensate for an uncontrolled foundation.

The work should remain proportionate. A POC foundation may be deliberately narrow and temporary if its limits are visible. An MVP or pilot foundation should be repeatable, testable, versioned and owned.

4.4. Practitioner note

The foundation does not need to be large to be valuable. It needs to be controlled, repeatable and able to scale.

A small set of curated assets with clear metrics, safe joins, enforced security rules, quality checks and known caveats is a better T2D foundation than broad access to raw or loosely documented data. The assistant should be innovative in the user experience, not improvisational in the data foundation.

Scalability should be designed into the foundation early. Reusable metadata templates, consistent naming, versioned logic, automated tests and code-based environment deployment can save significant time when moving from POC to MVP, pilot or production.

The balance is project-dependent. Too much engineering too early can slow learning; too little control can create foundations that cannot be trusted or scaled. This is where delivery experience and judgement matter: the team should build enough structure to protect quality, security and future scale, without turning a focused first release into an enterprise transformation.

5. Core foundation build activities

This chapter describes the core activities used to build the governed foundation. The activities should remain proportionate to the delivery stage. A POC may use lightweight controls to support learning and sizing. An MVP, pilot or production path requires stronger implementation discipline, security controls, quality checks, versioning and ownership.

Each activity should produce enough test evidence, logs, cost visibility and ownership records for the delivery stage. These controls are defined in Section 4.2 and should not be treated as optional administration.

5.1. Confirm foundation build scope

Purpose: Confirm what will actually be built in Phase 3.

This activity should normally be quick and straightforward. Phase 2 should already have identified the scope. The goal here is to identify any new issues exposed during build planning.

Core activities

- Review the Phase 2 answerability matrix and confirm which questions are included in the current build.
- Confirm which sources, tables, views, models, metrics and dimensions will be implemented.
- Identify which questions or assets are excluded, deferred or dependent on remediation.
- Confirm the target users and delivery stage: POC, MVP, pilot or production path.
- Confirm a named contact point or owner for each key source, metric, dimension and business question.
- Check whether implementation planning has exposed any new blockers, such as missing data, unsafe joins, contested logic, unclear ownership or security constraints.

Main outputs

- Confirmed foundation build scope.
- List of included questions, metrics, dimensions and source assets.
- List of exclusions, deferred items and remediation items.
- Named owners and operational contact points for the included assets and domains.
- Updated assumptions or constraints discovered during implementation planning.

Red flags

- The team starts building assets that were not assessed in Phase 2.
- The build scope expands because additional data is available, not because it is needed.
- Questions marked as caveated or unresolved are treated as ready.
- No one can clearly explain which questions the foundation is meant to support.
- The team does not know who can approve, clarify or fix key sources, definitions, caveats or quality issues.

5.2. Confirm implementation pattern and technology route

Purpose: Decide how the governed foundation will be implemented.

Core activities

- Confirm whether the foundation will use curated views, transformation models, semantic-layer objects, materialised tables, controlled APIs or a hybrid pattern.
- Identify the main technologies or platform components for storage, transformation, semantic logic, access control, testing and deployment.
- Confirm required skills, environments, deployment route and initial sizing.
- Ensure the pattern supports versioning, automated testing, CI/CD, rollback and controlled change.
- Assess implications for performance, refresh frequency, run cost, security and future reuse.
- Document unresolved technology decisions, assumptions or dependencies.
- Confirm whether architecture, security or finance approval is required, and whether an informal architecture steer is enough to begin controlled work.

Main outputs

- Agreed foundation implementation pattern.
- Named technologies or platform components.
- Initial sizing for build effort, run cost and scaling path.
- Key assumptions on performance, security, maintainability and scalability.
- Open technology decisions, owners and target resolution dates.
- Architecture / platform approval route, including informal steer and formal decision needs.
- Decision rationale where the default enterprise stack is not used.
- Automated testing approach and deployment validation route.

Red flags

- The pattern is chosen because it is quickest, not because it is controllable.
- The team starts building before the foundation technology route is clear.
- The chosen approach cannot support versioning, automated testing, deployment control or rollback.
- Security, performance or run-cost implications are unknown.
- The implementation pattern works for a demo but cannot scale to repeated use.
- The default enterprise stack is bypassed without a clear rationale.
- Build starts with no architecture steer or unclear formal approval route.

Practitioner note

Phase 3 should normally work within the enterprise's existing data and analytics stack unless there is a clear blocker. Introducing a new platform is a delivery decision, not a shortcut: skills, costs, security, data access patterns and operating ownership all need to be resolved.

An early informal architecture steer may be enough to start controlled foundation work, provided the formal approval route, open conditions and decision owner are documented..

5.3. Build curated queryable assets

Purpose: Create the controlled data assets the assistant is allowed to query.

Core activities

- Build or adapt the approved tables, views, models, marts, semantic-layer objects, materialised tables or APIs required for the scoped questions.
- Reduce direct access to raw source tables where governed, reusable assets can be created.
- Apply the agreed implementation pattern and technology route from Activity 5.2.
- Ensure assets expose only the columns, grains, relationships and data scope needed for the current delivery stage.
- Align asset names, field names and descriptions with the business language users are expected to use.
- Record lineage, creation date, owner, operational contact, source dependencies, limitations and caveats for each asset.
- Build and test assets through scripted, repeatable deployment wherever possible, including isolated test environments where appropriate.

Main outputs

- Curated queryable assets ready for T2D use.
- Asset register with purpose, scope, lineage, creation date, dependencies, owners, contacts, caveats and maintenance route.
- Defined allowed data scope and excluded raw tables, fields or sources.
- Deployment scripts or code-based build route for created assets.
- Automated asset test results or validation evidence.

Red flags

- The assistant is given broad access to raw tables because curated assets are not ready.
- Assets expose more data than the scoped questions require.
- Users' business terms cannot be reliably mapped to asset names or fields.
- The same business concept is exposed through multiple inconsistent assets.
- Assets are created without lineage, ownership, caveats, repeatable deployment or basic automated tests.

Practitioner note

Practitioner note: The controlled query surface should be as simple as the use case allows. A small number of well-named, well-owned and well-documented assets is usually safer than broad access to the data estate.

Except for deliberately lightweight POCs, foundation assets (incl. environments) should be created and deployed through code where practical, with tests and isolated environments for safe change.

5.4. Implement metric logic

Purpose: Make approved metrics usable by the assistant.

Core activities

- Implement the approved metric definitions required for the scoped questions.
- Encode calculations, date fields, time windows, exclusions, filters and caveats.
- Confirm the metric grain and valid aggregation behaviour.
- Align metric names, labels, synonyms and descriptions with business language.
- Record metric owner, operational contact, creation date, source dependencies and lineage.
- Validate implemented outputs against trusted reports, finance reconciliations or SME-approved examples where available.
- Ensure metric logic is versioned, testable and deployable through the agreed change route.
- Add automated tests for calculations, mandatory filters, date logic and reconciliation tolerances where appropriate.

Main outputs

- Implemented metric logic for the scoped questions.
- Metric implementation cards with definitions, calculations, grain, lineage, owners, contacts, caveats and non-use cases.
- Approved metric registry listing metrics available to T2D, including names, labels, synonyms and descriptions.
- Validation evidence, automated test results and reconciliation outcomes.
- Versioning and change-control route for metric updates.

Red flags

- The assistant is expected to infer metric logic from documentation or prompts.
- The same metric has different definitions or implementations across assets or teams.
- Date logic, exclusions, caveats or aggregation behaviour remain analyst-dependent.
- Metric outputs do not reconcile with trusted references and the difference is unexplained.
- Metric logic changes without ownership, versioning, automated tests or review.

Practitioner note

Metric logic becomes real when it is implemented. A definition that looked agreed in Phase 2 may expose edge cases once users see the actual calculation, exclusions, date logic and caveats.

Metric logic should be expected to evolve. For MVP or pilot, the metric registry should be versioned and linked to implemented logic so changes can be reviewed, tested and traced.

5.5. Implement dimensions and hierarchies

Purpose: Make filtering, grouping and drill-down behaviour clear.

Core activities

- Implement the approved dimensions required for the scoped questions.
- Define hierarchies, such as country > region > store, product category > sub-category > product, or year > quarter > month.
- Confirm approved labels, synonyms and business-friendly names for each dimension.
- Define which metrics each dimension can be used with, and which combinations should be blocked, caveated or clarified.
- Confirm the grain, join path and relationship between dimensions and queryable assets.
- Record dimension owner, operational contact, lineage, creation date, source dependencies, caveats and non-use cases.
- Identify any security, masking or inference implications to be handled in Activity 5.8.
- Validate key breakdowns and add automated tests for keys, hierarchy integrity, duplicate labels, orphan records and metric-dimension compatibility where appropriate.

Main outputs

- Implemented dimensions and hierarchies.
- Dimension register with labels, synonyms, hierarchy, lineage, owners, contacts, caveats and non-use cases.
- Mapping between dimensions, metrics and queryable assets.
- Approved join paths, grain assumptions and metric-dimension compatibility rules.
- Security or inference implications requiring control in Activity 5.8.
- Validation evidence and automated dimension test results.
- Change-control route for dimension updates.

Red flags

- Users' business terms cannot be reliably mapped to approved dimensions.
- The same dimension has different labels or hierarchies across assets.
- The assistant can group by dimensions that are invalid for the selected metric.
- Dimension grain, hierarchy or join path is unclear.
- Dimension updates are made without ownership, testing, review or compatibility checks.

Practitioner note

Practitioner note: Dimensions are not harmless labels. A user may be allowed to see total revenue, but not revenue by customer, employee, legal entity, store or small territory.

The team should identify which dimensions affect row-level access, masking, aggregation thresholds or inference risk, and either control or exclude them. If a dimension allows users to isolate a sensitive group, account, person or commercially restricted segment, it needs explicit control or exclusion.

5.6. Implement join, grain and aggregation rules

Purpose: Defines how assets can be combined, at what level of detail, and how results can be aggregated.

Core activities

- Confirm the approved grain for each key asset, such as order line, customer, account, product, day, month or transaction.
- Define approved join paths between facts, dimensions, reference tables and semantic-layer objects.
- Document relationship types, such as one-to-one, one-to-many, many-to-one or many-to-many.
- Identify joins that are not allowed, unsafe or dependent on remediation.
- Define valid aggregation behaviour for key metrics and dimensions.
- Add automated tests for duplicate rows, missing joins, cardinality, grain preservation and aggregation totals where appropriate.
- Record owners, operational contacts, lineage, creation date and change route for join and grain rules.

Main outputs

- Approved join, grain and aggregation rules.
- Register of allowed, restricted and excluded join paths.
- Grain definitions for key assets and metrics.
- Metric aggregation rules and invalid aggregation cases.
- Automated join, grain and aggregation test results.
- Known limitations, caveats, owners and change-control route.

Red flags

- The assistant can generate joins that were not explicitly approved.
- Valid SQL can still duplicate, omit or misaggregate records.
- Grain is unclear or differs across assets without explicit handling.
- Many-to-many relationships are exposed without controls or caveats.
- Aggregation rules are assumed rather than implemented, tested and owned.

Practitioner note

Many wrong T2D answers will not come from the model misunderstanding the question, but from the system using a plausible join or aggregation that is structurally wrong.

Join and aggregation rules are also security controls. A poorly controlled join can expose data a user should not see, bypass row-level restrictions, or allow sensitive information to

5.7. Implement standard filters and caveats

Purpose: Ensures that standard exclusions, time windows, default filters and known caveats are implemented in the governed foundation.

Core activities

- Identify standard filters required for the scoped questions, such as date windows, active records, completed transactions, valid products, regions or customer segments.
- Implement mandatory exclusions, such as test records, cancelled orders, internal users, inactive entities or non-reportable transactions.
- Confirm when filters should be applied by default and when the assistant should ask the user to clarify.
- Link filters and caveats to the relevant assets, metrics, dimensions and priority questions.
- Define caveats that should be surfaced in answers, such as provisional periods, delayed refunds, incomplete history or reconciliation limits.
- Record owner, operational contact, lineage, creation date, version and change route for standard filters and caveats.
- Validate key filters and caveats against trusted reports, SME examples or expected-answer cases.
- Add automated tests for mandatory exclusions, date logic, default filters and caveat linkage where appropriate.

Main outputs

- Implemented standard filters, exclusions and caveats.
- Filter and caveat register linked to assets, metrics, dimensions and priority questions.
- Default filter rules and clarification rules.
- User-facing caveat wording where relevant.
- Validation evidence and automated filter test results.
- Ownership, versioning and change-control route for filter and caveat updates.

Red flags

- Standard filters live only in analyst judgement, dashboard logic or prompt instructions.
- The assistant is expected to decide exclusions from ambiguous user wording.
- Different assets apply different filters for the same business question.
- Caveats are known but not linked to the relevant answer, metric or asset.
- Filters or caveats change without ownership, testing, versioning or review.

Practitioner note

Filters and caveats are often where a technically correct query becomes a wrong business answer. A query can use the right source, metric and join but still mislead users if it includes cancelled records, uses the wrong date field, ignores late-arriving data or hides a provisional-period caveat.

Clarification rules will never be exhaustive. The goal is to define the most common ambiguity patterns, such as unclear time periods, ambiguous business terms, missing filters or sensitive drill-downs, and give the assistant clear default behaviours: apply the approved default, ask a clarification question, surface a caveat, suppress the answer or escalate.

5.8. Implement security and exposure controls

Purpose: Control what users can see, query or infer.

Core activities

- Define the user groups or roles in scope and map each group to the assets, metrics, dimensions, rows, columns and aggregations they are allowed to access.
- Identify sensitive assets, fields, dimensions, metrics and combinations exposed through the governed foundation.
- Implement row-level, column-level, masking and aggregation controls required for the scoped users.
- Define minimum group-size rules, suppression rules or drill-down limits where small populations could expose sensitive information.
- Confirm which joins, dimensions or filters have security implications and link them to the relevant controls.
- Define audit requirements for access changes, denied access, sensitive queries, security-rule changes and exposure events.
- Validate security behaviour with representative user roles, including allowed access, denied access, masked outputs and restricted drill-downs.
- Add automated tests for security policies, masking, aggregation thresholds and role-based access where appropriate.
- Record control owner, operational contact, version, effective date, approval route and unresolved security assumptions.
- Apply request and result-level security checks where appropriate, including validation of the generated query or API request before execution and inspection of returned results before they are exposed to the user.

Main outputs

- Implemented security and exposure controls.
- Security control register covering row, column, masking, aggregation and inference controls.
- Mapping between controls, users, assets, metrics, dimensions, joins and filters.
- Audit and logging requirements for security-relevant events.
- Validation evidence and automated security test results.
- Known residual risks, assumptions and security handover items.
- Ownership, versioning and change-control route for security controls.
- User group and access matrix for the governed foundation.
- Request and result-level security checks covering generated queries, API requests, returned fields, small cohorts, masking behaviour, aggregation limits and restricted result shapes.

Red flags

- The team treats authentication as sufficient security.
- Users can drill down, filter or join data in ways that expose restricted groups or sensitive information.
- Sensitive fields are hidden in one asset but exposed through another route.
- Access rules are documented but not implemented or tested in the foundation.
- Security exceptions are handled manually or informally without approval, audit or expiry.
- No one can approve residual exposure risk for the intended delivery stage.
- The generated query is checked before execution, but the returned result is not assessed before being shown to the user.

Practitioner note

Practitioner note: T2D security should not create a parallel access model unless there is a clear reason. User groups should reuse existing enterprise groups, roles or identity structures where possible, because access rules often depend on reporting lines, region assignments, legal entities, cost centres or commercial territories.

T2D security is broader than dashboard access. Users may infer restricted information through repeated questions, narrow filters, small-group aggregations or unsafe joins, even when direct column access is blocked.

5.9. Add quality, performance, freshness and cost controls

Purpose: Make sure the foundation can be trusted, refreshed and afforded in use.

Core activities

- Define quality checks for key assets, metrics, dimensions, joins and filters.
- Define freshness expectations, including refresh frequency, data latency and acceptable delay.
- Implement reconciliation checks against trusted sources, reports or SME-approved examples where relevant.
- Identify performance expectations for priority queries, including runtime, concurrency and response-time needs.
- Track expected run-cost drivers, separating:
    - **Infrastructure costs** such as storage, compute, refresh, materialisation, testing, monitoring and query execution
    - **Support costs** such as incident handling, access requests, metric changes, quality remediation and ongoing ownership.
- Define cost thresholds and alert routes for unusual spend, expensive queries, excessive refresh cost or unexpected storage growth.
- Record owners, operational contacts, version, review frequency and escalation route for quality, freshness, performance and cost controls.
- Add automated tests and alerts where appropriate, with evidence centralised as defined in Section 4.2.

Main outputs

- Quality, freshness, performance and cost-control definitions.
- Implemented quality checks, freshness checks and reconciliation checks.
- Performance assumptions and priority-query benchmarks.
- Cost-tracking view by meaningful foundation layer.
- Alert thresholds and escalation routes for quality, freshness, performance and cost issues.
- Validation evidence, automated test results and cost observations.
- Ownership, versioning and change-control route for control updates.

Red flags

- Data can be queried even when freshness or quality status is unknown.
- Priority queries are too slow or expensive for repeated use.
- Cost is tracked only at aggregate level, or only for infrastructure, with no view of support effort or scaling drivers.
- Quality, freshness or cost alerts go to no clear owner.
- Reconciliation gaps are known but not surfaced, owned or caveated.
- Controls work for a demo but cannot support MVP, pilot or production usage.

Practitioner note

Cost assessment will be approximate at this stage because usage patterns, query volume, refresh frequency and support needs are still uncertain. The aim is not to produce a perfect run-cost model, but to identify the main cost drivers and scaling risks early.

A practical approach is to estimate a few scenarios, such as narrow POC usage, expected MVP usage and higher-volume pilot usage, separating infrastructure costs from support costs.

5.10. Package foundation for handover

Purpose: Make the foundation usable beyond the build team.

Core activities

- Consolidate the approved assets, metrics, dimensions, joins, filters, caveats, security controls and quality controls into a governed foundation pack.
- Confirm that registers are complete enough for the intended delivery stage, including owners, operational contacts, versioning, lineage and change routes.
- Centralise test evidence, validation results, logs, cost observations, known failures and unresolved issues.
- Document limitations, residual risks, exclusions, deferred items and remediation actions.
- Confirm the support route for incidents, access questions, data-quality issues, metric changes and user feedback.
- Define how foundation changes will be requested, reviewed, tested, released and communicated.
- Confirm what needs to be handed over to architecture, orchestration, evaluation, security validation, adoption and operations.

Main outputs

- Governed foundation pack.
- Completed registers for assets, metrics, dimensions, joins, filters, security controls and quality/cost controls.
- Centralised test results, validation evidence, logs and cost observations.
- Known limitations, residual risks, exclusions and remediation backlog.
- Support and escalation route.
- Change-control and release route.
- Handover notes for architecture, orchestration, evaluation, security and operations.

Red flags

- Knowledge of the foundation sits mainly with individual builders.
- Registers exist but are incomplete, inconsistent or not linked to implemented assets.
- Test results, logs, costs and known issues are scattered across tools or not retained.
- There is no clear owner for incidents, access questions, metric changes or quality issues.
- Later phases need to rediscover what is approved, caveated, excluded or unsafe.

Annex reference

See annex for a governed foundation handover checklist covering registers, evidence, logs, cost observations, owners, support route, change control, known limitations and downstream handover

Practitioner note

The foundation is not finished when the assets deploy. It is finished when another team can understand, test, use, govern and operate it.

6. Governed foundation decision pack

Phase 3 should not end with a collection of disconnected assets, notes and technical artefacts. It should end with a consolidated foundation pack that shows what has been built, what is approved, what is tested, what is caveated, what remains unresolved and who owns each material item.

The output should be proportionate to the delivery stage. For a POC, it may be lightweight and focused on learning, feasibility and MVP sizing. For an MVP, pilot or production path, the outputs should be more formal, versioned, tested and suitable for handover.

6.1. Governed foundation pack

Field	Purpose
Foundation scope	Defines the users, questions, metrics, dimensions and domains supported by the foundation.
Approved queryable assets	Lists the tables, views, models, semantic-layer objects, materialised tables or APIs the assistant may query.
Implementation pattern and technology route	Records the agreed foundation pattern, technologies used and unresolved platform decisions.
Metric registry	Lists approved metrics, definitions, implementation references, synonyms, caveats, owners and versions.
Dimension and hierarchy register	Documents approved dimensions, hierarchies, synonyms, valid metric combinations and security implications.
Join, grain and aggregation rules	Defines approved joins, asset grains, aggregation behaviour and restricted or excluded combinations.
Filter and caveat register	Records mandatory filters, default exclusions, clarification rules and caveats to surface in answers.
Security and exposure controls	Defines user groups, access rules, masking, aggregation thresholds, inference controls and residual risks.
Quality, freshness, performance and cost controls	Captures checks, thresholds, alerts, benchmarks, cost scenarios and cost reporting layers.
Test, log and evidence record	Centralises automated test results, validation evidence, logs, approvals and unresolved failures.
Ownership and operating route	Names owners, operational contacts, escalation routes and change-control paths.
Known limitations and backlog	Lists deferred items, remediation actions, exclusions and risks that remain outside the current foundation.

The pack should clearly identify the metadata contract required for Phase 4 orchestration, including which artefacts are structured enough for retrieval, validation, query generation and answer explanation.

6.2. Foundation pack quality test

Before Phase 3 exits, the team should test whether the consolidated outputs are usable by later phases.

Quality test	Question
Scope clarity	Is it clear which questions, users, metrics, dimensions and assets are supported?
Queryability	Are the approved assets actually available for the intended T2D architecture to query?
Semantic consistency	Are metrics, dimensions, filters and caveats implemented consistently across assets?
Safety	Are access, masking, aggregation and inference controls defined, implemented and tested where required?
Traceability	Can the team trace an answer back to the asset, metric version, filters, joins, caveats and tests used?
Test evidence	Are automated tests, manual validations and reconciliation evidence centralised and linked to the relevant artefacts?
Cost visibility	Are expected infrastructure and support costs visible by meaningful layer and scenario?
Operational ownership	Does every material asset, rule, caveat, control and known issue have an owner or decision route?
Change control	Is there a clear route to update, test, approve, deploy and roll back foundation changes?
Handover readiness	Can architecture, build, evaluation, security validation and operations use the pack without rediscovering the foundation?

A good Phase 3 output is not necessarily large. It is clear, controlled, testable and usable. If later phases cannot understand what the assistant may query, which rules apply, what was tested, what risks remain or who owns changes, Phase 3 has not finished.

7. Exit criteria and handover

Phase 3 should exit only when the governed foundation is controlled enough for the intended next step. The handover should confirm what has been built, what has been tested, what remains limited, and which risks or remediation items still need ownership.

A Phase 3 exit does not mean the full T2D product is ready for launch. It means the foundation is ready to be used by the next delivery phases.

7.1. Required exit outputs

The required exit output is the governed foundation decision pack described in Section 6, completed to the standard required for the delivery stage.

The exit decision should confirm whether the foundation can proceed, narrow, remediate, defer or stop. It should also identify which risks, limitations, remediation items and ownership gaps remain open.

7.2. Handover to later phases

Later phase	What Phase 3 should hand over
Architecture and orchestration design	Approved assets, access patterns, metadata sources, query boundaries, performance assumptions and safe-failure needs.
Prototype / MVP build	Queryable assets, implemented metrics, dimensions, joins, filters, caveats and usage constraints.
Evaluation	Expected-answer sources, validation evidence, known caveats, failure cases and regression-test inputs.
Security validation	User groups, access controls, masking rules, aggregation thresholds, audit needs and residual risks.
Adoption	User-facing scope, supported questions, known caveats, exclusions and escalation routes.
Operations	Owners, contacts, logs, alerts, quality checks, cost tracking, support route and change-control process.
Orchestration-ready metadata	Phase 4 can identify which artefacts are runtime-ready and which remain manual, incomplete or unsuitable.

8. Key risks and failure modes

Risk / failure mode	Why it matters	Likely response
Raw data access becomes the shortcut	The assistant may query unmanaged tables, inconsistent fields or sensitive data outside the governed foundation.	Narrow access to approved queryable assets; exclude raw sources unless explicitly approved and controlled.
Business logic remains in prompts	Metrics, filters, joins or caveats may be applied inconsistently or lost when prompts change.	Encode logic in governed assets, semantic objects or registries wherever possible.
Metric implementation drifts from the approved definition	The assistant may appear grounded while using an outdated or incorrect calculation.	Version metric definitions and implementations; require review, tests and reconciliation evidence.
Dimensions enable unsafe drill-down	Users may access or infer restricted data by slicing results through sensitive dimensions.	Define valid dimensions, aggregation thresholds, masking rules, role-based restrictions.
Joins create wrong or unsafe answers	Valid SQL can duplicate, omit or expose records if grain and relationships are not controlled.	Approve join paths, document grain, test cardinality and exclude unsafe joins.
Filters and caveats are not applied consistently	Answers may include cancelled records, wrong periods, incomplete data or hidden limitations.	Implement standard filters, default rules, caveat triggers and clarification behaviours.
Security is treated as authentication only	Users may still infer restricted information through joins, filters, aggregations or repeated questions.	Implement exposure controls: row, column, masking, aggregation, inference and audit rules.
Quality or freshness state is invisible	The system may answer confidently from stale, incomplete or unreconciled data.	Add freshness checks, quality thresholds, reconciliation checks, alerts and answer caveats.
Foundation cost is not understood	A technically correct foundation may become too expensive to refresh, test, monitor or query.	Track costs by layer and scenario; set thresholds and review cost drivers before scaling.
Manual build steps create fragile foundations	Assets may be hard to reproduce, test, review or roll back.	Use scripted deployment, CI/CD, automated tests and code-based environments where practical.
Ownership is unclear after build	Issues, changes and failures may not be resolved once the implementation team moves on.	Assign owners, operational contacts, support routes and change-control paths before exit.
The foundation is too broad for the delivery stage	Trying to support too many domains, questions or users may weaken control and delay learning.	Narrow the scope to the smallest controlled foundation that supports the next decision.